NAME OF THE STUDENT : P. MUNI KARTHIK

REGISTRATION NUMBER : 192425061

COURSE CODE : DSA0306

COURSE NAME : NATURAL LANGUAGE PROCESSING

ASSESMENT NUMBER : CO2 AT-2

A company is developing an NLP-based grammar checker for educational applications. The system is trained

using a manually POS-tagged corpus. Your task is to recreate the Hidden Markov Model (HMM) without using

any built-in NLP or HMM libraries.

From the given corpus,

- determine the Emission Probability
- determine the Transition Probability
- implement the Viterbi Algorithm
- predict the POS tags for the new input sentence.

Use only Python dictionaries, loops, and basic programming constructs.

Training Corpus

Sentence POS Tags

The/DT boy/NN eats/VBZ rice/NN DT NN VBZ NN

The/DT girl/NN drinks/VBZ milk/NN DT NN VBZ NN

A/DT cat/NN drinks/VBZ milk/NN DT NN VBZ NN

The/DT dog/NN chases/VBZ cat/NN DT NN VBZ NN

A/DT teacher/NN teaches/VBZ students/NNS DT NN VBZ NNS

Students/NNS study/VBP English/NN NNS VBP NN

Birds/NNS fly/VBP high/RB NNS VBP RB

Children/NNS play/VBP games/NNS NNS VBP NNS

New Input Sentence

The cat drinks milk

Tasks

1. Separate each sentence into words and POS tags.

2. Compute the Emission Probability of every word.

3. Compute the Transition Probability between POS tags.

4. Recreate the Hidden Markov Model using Python dictionaries.

5. Implement the Viterbi Algorithm without using any built-in HMM/NLP libraries.

6. Predict the POS tag sequence for “The cat drinks milk”

Q1 – Hidden Markov Model (HMM) and Viterbi Algorithm

```
# ------------------------------
# Training Corpus
# ------------------------------

corpus = [
    [("The","DT"),("boy","NN"),("eats","VBZ"),("rice","NN")],
    [("The","DT"),("girl","NN"),("drinks","VBZ"),("milk","NN")],
    [("A","DT"),("cat","NN"),("drinks","VBZ"),("milk","NN")],
    [("The","DT"),("dog","NN"),("chases","VBZ"),("cat","NN")],
    [("A","DT"),("teacher","NN"),("teaches","VBZ"),("students","NNS")],
    [("Students","NNS"),("study","VBP"),("English","NN")],
    [("Birds","NNS"),("fly","VBP"),("high","RB")],
    [("Children","NNS"),("play","VBP"),("games","NNS")]
]

# ------------------------------
# Step 1: Separate Words and Tags
# ------------------------------

print("Words and POS Tags:\n")
```

```
for sentence in corpus:
    words = []
    tags = []
    for word, tag in sentence:
        words.append(word)
        tags.append(tag)

    print("Words :", words)
    print("Tags  :", tags)
    print()

# ------------------------------
# Step 2: Emission Probability
# ------------------------------

emission_count = {}
tag_count = {}

for sentence in corpus:
    for word, tag in sentence:

        if tag not in emission_count:
            emission_count[tag] = {}

        if word not in emission_count[tag]:
            emission_count[tag][word] = 0
```

```
        emission_count[tag][word] += 1

        if tag not in tag_count:
            tag_count[tag] = 0

        tag_count[tag] += 1

print("\nEmission Probabilities\n")

emission_prob = {}

for tag in emission_count:

    emission_prob[tag] = {}

    for word in emission_count[tag]:

        emission_prob[tag][word] = emission_count[tag][word] / tag_count[tag]

        print(tag,"->",word,"=",round(emission_prob[tag][word],3))

# ------------------------------
# Step 3: Transition Probability
# ------------------------------

transition_count = {}
```

```
transition_total = {}

for sentence in corpus:

    tags = []

    for word, tag in sentence:
        tags.append(tag)

    for i in range(len(tags)-1):

        t1 = tags[i]
        t2 = tags[i+1]

        if t1 not in transition_count:
            transition_count[t1] = {}

        if t2 not in transition_count[t1]:
            transition_count[t1][t2] = 0

        transition_count[t1][t2] += 1

        if t1 not in transition_total:
            transition_total[t1] = 0

        transition_total[t1] += 1
```

```
print("\nTransition Probabilities\n")

transition_prob = {}

for t1 in transition_count:

    transition_prob[t1] = {}

    for t2 in transition_count[t1]:

        transition_prob[t1][t2] = transition_count[t1][t2] / transition_total[t1]

        print(t1,"->",t2,"=",round(transition_prob[t1][t2],3))

# ------------------------------
# Step 4: Hidden Markov Model
# ------------------------------

print("\nHidden Markov Model\n")

HMM = {
    "Emission": emission_prob,
    "Transition": transition_prob
}

print(HMM)
```

```
# ------------------------------
# Step 5: Viterbi Algorithm
# ------------------------------

sentence = ["The","cat","drinks","milk"]

states = list(emission_prob.keys())

V = []
path = {}

# Initial Step

V.append({})

for state in states:

    emit = emission_prob[state].get(sentence[0],0)

    V[0][state] = emit

    path[state] = [state]

# Recursion

for t in range(1,len(sentence)):
```

```
V.append({})
newpath = {}

for curr in states:

    best_prob = 0
    best_state = None

    emit = emission_prob[curr].get(sentence[t],0)

    for prev in states:

        trans = transition_prob.get(prev,{}).get(curr,0)

        prob = V[t-1][prev] * trans * emit

        if prob > best_prob:
            best_prob = prob
            best_state = prev

    V[t][curr] = best_prob

    if best_state != None:
        newpath[curr] = path[best_state] + [curr]

path = newpath
```

```
# Final Step

best_prob = 0
best_state = None

for state in states:

    if V[-1][state] > best_prob:
        best_prob = V[-1][state]
        best_state = state

print("\nPredicted POS Tags\n")

print("Sentence :",sentence)
print("Tags     :",path[best_state])
```

OUTPUT :

```
Words and POS Tags:

Words : ['The', 'boy', 'eats', 'rice']
Tags  : ['DT', 'NN', 'VBZ', 'NN']

Words : ['The', 'girl', 'drinks', 'milk']
Tags  : ['DT', 'NN', 'VBZ', 'NN']

Words : ['A', 'cat', 'drinks', 'milk']
Tags  : ['DT', 'NN', 'VBZ', 'NN']

Words : ['The', 'dog', 'chases', 'cat']
Tags  : ['DT', 'NN', 'VBZ', 'NN']

Words : ['A', 'teacher', 'teaches', 'students']
Tags  : ['DT', 'NN', 'VBZ', 'NNS']

Words : ['Students', 'study', 'English']
Tags  : ['NNS', 'VBP', 'NN']

Words : ['Birds', 'fly', 'high']
Tags  : ['NNS', 'VBP', 'RB']

Words : ['Children', 'play', 'games']
Tags  : ['NNS', 'VBP', 'NNS']


Emission Probabilities

DT -> The = 0.6
DT -> A = 0.4
NN -> boy = 0.1
NN -> rice = 0.1
NN -> girl = 0.1
NN -> milk = 0.2
NN -> cat = 0.2
NN -> dog = 0.1
NN -> teacher = 0.1
NN -> English = 0.1
VBZ -> eats = 0.2
VBZ -> drinks = 0.4
VBZ -> chases = 0.2
VBZ -> teaches = 0.2
```

```
NNS -> students = 0.2
NNS -> Students = 0.2
NNS -> Birds = 0.2
NNS -> Children = 0.2
NNS -> games = 0.2
VBP -> study = 0.333
VBP -> fly = 0.333
VBP -> play = 0.333
RB -> high = 1.0

Transition Probabilities

DT -> NN = 1.0
NN -> VBZ = 1.0
VBZ -> NN = 0.8
VBZ -> NNS = 0.2
NNS -> VBP = 1.0
VBP -> NN = 0.333
VBP -> RB = 0.333
VBP -> NNS = 0.333

Hidden Markov Model

{'Emission': {'DT': {'The': 0.6, 'A': 0.4}, 'NN': {'boy': 0.1, 'rice': 0.1, 'girl': 0.1, 'milk': 0.2, 'cat': 0.2, 'dog': 0.1, 'teacher': 0.1, 'English': 0.1}, 'VBZ': {'eats': 

Predicted POS Tags

Sentence : ['The', 'cat', 'drinks', 'milk']
Tags     : ['DT', 'NN', 'VBZ', 'NN']
```

Manual Answer for Record

1. Separate each sentence into words and POS tags

Sentence	POS Tags
The boy eats rice	DT NN VBZ NN
The girl drinks milk	DT NN VBZ NN
A cat drinks milk	DT NN VBZ NN
The dog chases cat	DT NN VBZ NN
A teacher teaches students	DT NN VBZ NNS
Students study English	NNS VBP NN
Birds fly high	NNS VBP RB
Children play games	NNS VBP NNS

2. Emission Probability (Examples)

DT

- P(The|DT)=3/5=0.60
- P(A|DT)=2/5=0.40

NN

- boy=1/10=0.10
- girl=1/10=0.10
- cat=2/10=0.20
- milk=2/10=0.20
- rice=1/10=0.10
- dog=1/10=0.10
- teacher=1/10=0.10
- English=1/10=0.10

VBZ

- drinks=2/5=0.40
- eats=1/5=0.20
- chases=1/5=0.20
- teaches=1/5=0.20

NNS

- students=1/5=0.20
- Students=1/5=0.20
- Birds=1/5=0.20
- Children=1/5=0.20
- games=1/5=0.20

VBP

- study=1/3=0.333
- fly=1/3=0.333
- play=1/3=0.333

RB

- high=1/1=1.00

3. Transition Probability

Transition	Probability
DT → NN	5/5 = 1.0
NN → VBZ	5/10 = 0.5
NN → NNS	1/10 = 0.1
VBP → NN	1/3 = 0.333
VBP → RB	1/3 = 0.333
VBP → NNS	1/3 = 0.333
VBZ → NN	4/5 = 0.8
VBZ → NNS	1/5 = 0.2
NNS → VBP	3/5 = 0.6

4. Hidden Markov Model

- States: DT, NN, VBZ, NNS, VBP, RB
- Emission Matrix: Computed above
- Transition Matrix: Computed above

5. Viterbi Algorithm

The algorithm computes the most probable sequence of POS tags using:

- Emission Probabilities
- Transition Probabilities
- Dynamic Programming (Viterbi)

6. Predicted POS Tags

Input Sentence

The cat drinks milk

Predicted POS Tags

DT NN VBZ NN

This solution is appropriate for your assessment because it recreates the HMM from scratch using only dictionaries, loops, and basic Python constructs, with no built-in NLP or HMM libraries.